

Banking :

Multi-Tier-With Database: **MYSQL** database

Start + Project Repo

- + Setup **EKS** cluster **Using Terraform**,
- + Create **VM** & Setup **SonarQube** and **Nexus**
- + **Setup Jenkins** & **Install Required Plugins**
- + Create a Complete CICD pipeline
- + **Custom Domain** mapping using **DNS in GoDaddy**
- + **SSL CERTIFICATE** Configuration

Most important

Make sure you are in **main branch** in **Git** than go to **SRC** folder Go to Main here **you find Resources folder** inside which we have **application. Properties**.

These **file contain configuration** about the **database**. So when **we are deploy our applciation** inside K8's we can take use these details. Just remember that .

Lets **Create Instance** : 3 server t2 medium

Which will be like a Central server.

Instance type : t.medium

Security group : these are the **Ports Open**

Inbound rules (7)							
Search							< 1
Security group rule...	IP version	Type	Protocol	Port range	Source		
sg-07a0efc77431717af	IPv4	SMTP	TCP	25	0.0.0.0/0		
sg-0a6665c912070da...	IPv4	HTTP	TCP	80	0.0.0.0/0		
sg-05590fee51e1ca1a8	IPv4	HTTPS	TCP	443	0.0.0.0/0		
sg-00ed6309b58cd34...	IPv4	Custom TCP	TCP	2000 - 11000	0.0.0.0/0		
sg-03b2cd178fd75d5b7	IPv4	SSH	TCP	22	0.0.0.0/0		
sg-0e892ccde6f5805c6	IPv4	Custom TCP	TCP	6443	0.0.0.0/0		
sg-04a1464c0ba94b8...	IPv4	SMTPS	TCP	465	0.0.0.0/0		

Storage : **20 GB**

Login to server :

Sudo apt update

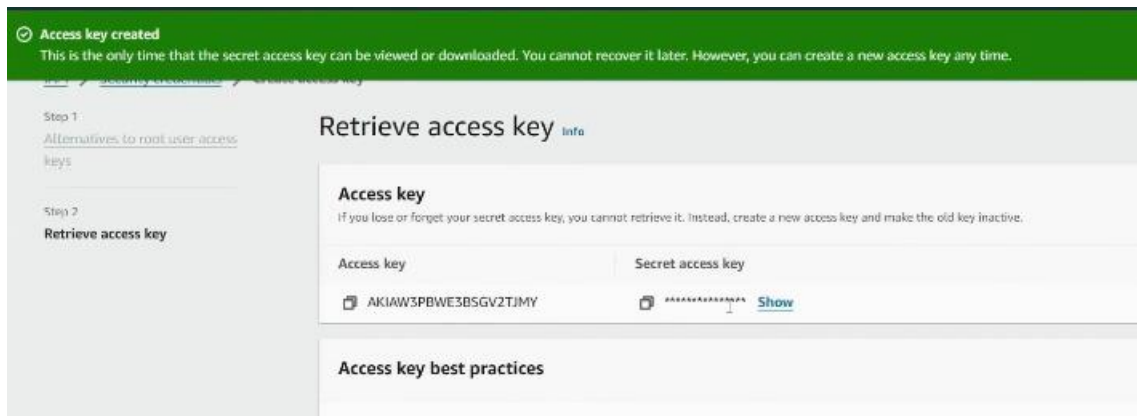
Using Terraform we **Create EKS** inside **AWS** for that **we want AwSCLI**

Install AWSCLI

```
curl
"https://awscli.amazonaws.com/awscli-exe-lin
zip" -o "awscliv2.zip"
sudo apt install unzip
unzip awscliv2.zip
sudo ./aws/install
aws configure
```

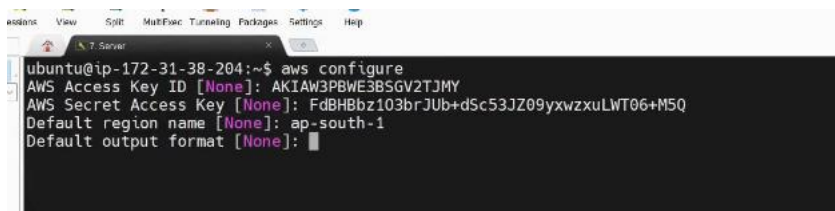
Copy these and run it in Server.

Either you can create specific **user** in **IAM** provide certain **role** and then use its credential . for demo purpose I go with my **root account** , click on security credentials + **create access key** in case if you don't want to forget you can download in CSV file.



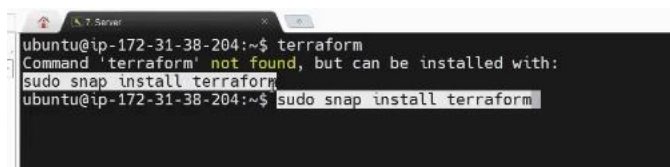
Create IAM user.

We use **accesskey** and **secret key**.



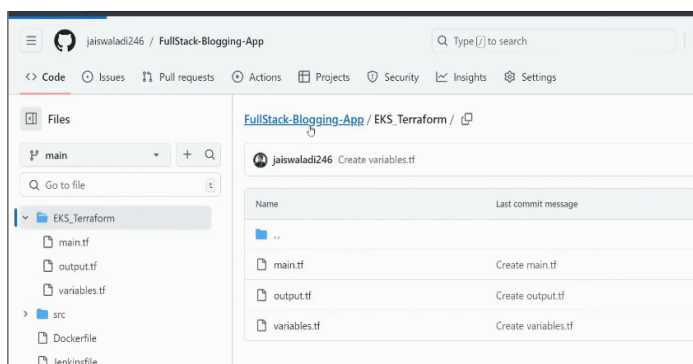
Now we can run **terraform** command to **create the EKS cluster** for that make sure **terraform** install.

Terraform Install :

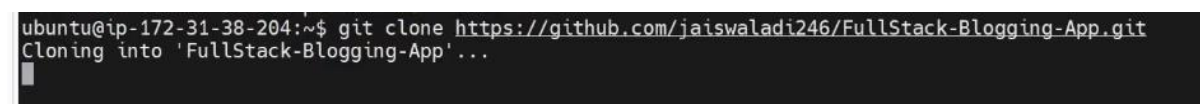


Make sure you have to add **--classic**

How you get it script **terrform** .



Simply **clone these Repository** :



Now we go inside EKS folder :

```
ubuntu@ip-172-31-38-204:~$ cd FullStack-Blogging-App/EKS_Terraform/  
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$
```

```
Sessions View Split MultiExec Tunneling Packages Settings Help  
7. Server  
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$ ls  
main.tf output.tf variables.tf  
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$
```

So every thing in these terraform file are added in variable formate No hard coded is done

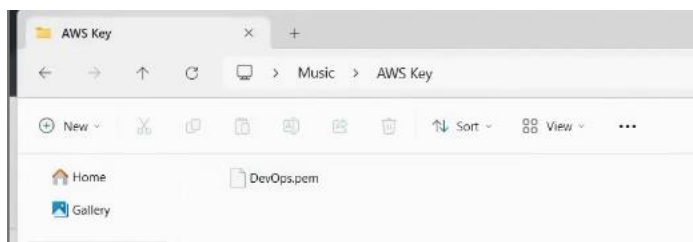
Region :

Availability zone : based on as same as your region

After that you have to Go to variable .

```
FullStack-Blogging-App / EKS_Terraform / variables.tf  
jaiswaladi246 Create variables.tf  
Code Blame 5 lines (5 loc) · 144 Bytes  
1 variable "ssh_key_name" {  
2   description = "The name of the SSH key pair to use for instances"  
3   type       = string  
4   default    = "DevOps"  
5 }
```

You Need to provide **Private Key Name** "DevOps".



Now we run **terraform init**

Terraform plan (I will tell you how many resources created)

Final command to **create resources** : Terraform apply **-auto-approve**

Now let say I want to access our cluster how we can you make sure that **kubectI** present on machine

Cluster is created **Connection refuse** because **we have not yet connect to EKS cluster** for authentication
kuberneates cluster we need **kube config file** that **contain configuration** and everthing .

We provide **cluster name** also .

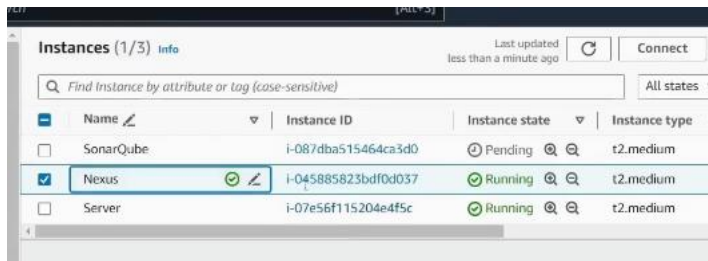
```
aws eks --region ap-south-1 update-kubeconfig --name  
devopsshack-cluster
```

```
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-0-0-119.ap-south-1.compute.internal Ready    <none>   7m13s v1.30.2-eks-1552ad0
ip-10-0-1-133.ap-south-1.compute.internal Ready    <none>   7m16s v1.30.2-eks-1552ad0
ip-10-0-1-217.ap-south-1.compute.internal Ready    <none>   7m15s v1.30.2-eks-1552ad0
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$
```

Here worker nodes are present .

Now we Go to **Ec2 instance** :

2 instance : T2.medium : Storage : **20 GB**



1. SonarQube
2. Nexus we name it server.

Lets connect with sonarqube server

Copy **Public IP address** Go to terminal similar we can do with nexus machine also

Now we setup these using **docker container** because that will be Easiest .

Now we are Going to setup sonarqube and Nexus in these virtual machine we are doing these with Using **Docker container** because that will be the **Easiest**.

1. System Update :

Here first thing I going to do is

\$ **Sudo apt update** <- this command you have to **run in every virtual machine for first time** .

So that **all the packages are updated** and they are available .

As I say we use docker to setup these tools so we need to **make sure that docker has installed** for that I can run docker.

2. Install Docker :

\$ **sudo apt install docker.io -y**

Similar we have to do these steps with **other machine** also **Sonarqube** .

So the thing is **when you install docker** by default **root user** has permission to run theses Docker command

In case you want to **your current user** to run docker command.

you have Two option

1. Option we can give all permission to other user or
2. we can give specific permission only to user.

You don't want to do these steps you can switch to Root user.

Virtual machine : container port

8081:8081

Nexus docker image install:

```
sions View Split MultiExec Tunneling Packages Settings Help
7. Server 8. Nexus 10. SonarQube
ubuntu@ip-172-31-33-18:~$ sudo docker run -d --name Nexus3 -p 8081:8081 sonatype/nexus3
Unable to find image 'sonatype/nexus3:latest' locally
```

Check status :

Sudo docker ps

Now I believe my nexus will be access it will be **port 8081**

<public ip >:8081

Here you can see Nexus is up

- **Sign-in**

- We have **setup Nexus using docker container** so **password** should be **inside these container**. So we have to go inside How we can Go

-it means interactive terminal we interact container through terminal.

Bash terminal

```
ubuntu@ip-172-31-33-18:~$ sudo docker exec -it bb8f60e4cc27 /bin/bash
bash-4.4$
```

```
bash-4.4$ ls
nexus sonatype-work start-nexus-repository-manager.sh
bash-4.4$
```

```
nexus sonatype-work start-nexus-repository-manager.sh
bash-4.4$ cd sonatype-work/nexus3/
bash-4.4$ ls
admin.password  cache  elasticsearch  generated-bundles  javaprefs  keystores  log  restore-from-backup
blobs           db      etc            instances          karaf.pid  lock       port  tmp
bash-4.4$
```

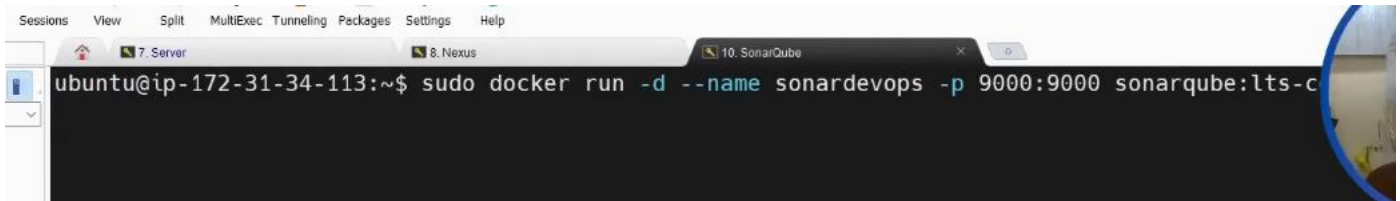
```
bash-4.4$ cat admin.password
a8dc4efd-e335-42e0-8195-c73168aaa2c2bash-4.4$
```

Put it in your nexus

Now we go to **sonarQube** virtual machine :

Here we install **apt get update** , docker , download sonarqube docker image

Sudo we use because we **do not want to give permission other user**



<IP>:9000

Username: Admin

Password: admin

Jenkins - virtual machine

we created and these machine will be **big because we don't have slave machine** all **job will run in these master machine** . **t2 large**

Q. interview Question **how we setup slave machine** to run or Job ?

Now **copy ip address jenkins** public just connect it Devopsshack using mobaxterm terminal

- **sudo apt update**

- for jenkins there is **pre requisite is install JAVA**

- install kubectl

We install open jdk 17

Google it jenkins in ubuntu install command copy it

By default jenkins is setup port 8080

Manage jenkins → Manage plugins →

1. Sonnarqube scanner
2. Config file provider
3. Maven integration
4. Pipeline maven integration
5. Kubernetes
6. Kubernetes CLI
7. Kubernetes credentials
8. Kubernetes Client API
9. Docker
10. Docker pipeline

Click on Install

Go back **manage jenkins** go to **tools**

- **Jdk** you don't have to install any saprate plugin one **jdk avilable by default** we use it

- Sonnar Qube scanner

- maven version select 3.99

Here **Jenkins Virtual machine** we also **install docker**

Here we **provide permission to user**

Now we **create pipelien in jenkins**

- **New item** + (name it Full stack)+ Pipeline
- **Discard old build** -2 build we can keep it.

Explain Jenkins File :

So we want **Multiple Stage** :

We know that **we** are **not install maven** in our **Virtual Machine** we **install plugin** at that time we have to **Mention** it in here **Tools pipeline Script**.

Maven3 is tool name



```
Script ?
1 pipeline {
2   agent any
3
4   tools {
5     maven 'maven3'
6   }
7 }
```

Here maven is **3rd party plugin** :

Maven is **not install** in **server** but we configure it inside tool section of jenkins in that case maven is 3rd party plugin. tools block tool **type = toolname** (maven3) now maven is available for usage .

Similarly,

we have to define **sonarqube scanner** tool You might thinking why we are not adding to these tool block sonarqube

tools are those are **managed by jenkins** and sonarqube scanner is not the tool which is managed by jenkins In that case we have to define sonarqube scanner in different ways we can define it using **environment variable**

we create a **variable** that should pointing to **home directory** of sonarqube tool (**tool name sonar-scanner**)



```
8 environment {
9   SCANNER_HOME= tool 'sonar-scanner'
10 }
11
```

Managed jenkins + System (inside we need to configure the server) + scroll down you able to find the sonarqube server click on add sonarqube installation

URL : Go to browser you see **sonarqube** runing on **IP: port** copy that and past it here

Dashboard > Manage Jenkins > System >

Name

sonar

Server URL

Default is http://localhost:9000

http://3.110.131.62:9000/

Where we getting **Credentials** for that you have to **Go to sonarqube dashboard** + administrator + security + user **Generate Token** give it name A COPY that

Kind : **Secret Text**

← → ↻ Not secure 13.233.155.197:8080/manage/credentials/store/system/domain/_/newCredentials

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) >

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

ID ?

sonar-token

Description ?

sonar-token

Create

← → ↻ Not secure 13.233.155.197:8080/manage/configure

Dashboard > Manage Jenkins > System >

Server URL

Default is http://localhost:9000

http://3.110.131.62:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-token

+ Add

Advanced

1. **GitCheckout** : Means we **Create Local Copy of Source Code** inside Jenkins

If you are New Go to **Pipeline Syntax** = Git: git , Repo URL , Branch , Credentials than Generate Syntax.

!!! Credentials in Git + jenkins pipeline !!!

```
Script ?
1 pipeline {
2   agent any
3
4   stages {
5     stage('Git Checkout') {
6       steps {
7         git branch: 'main', url: 'https://github.com/jaiswaladi246/Multi-Tier-With-Database.git'
8       }
9     }
10  }
```

Jenkins Credentials Provider: Jenkins

Username ?
jaiswaladi246

☐ Treat username as secret ?

Password ?
.....

ID ?
git-cred

Description ?
git-cred

2. **Compile** : Check Syntax based Error : [don't say in interview compile when mvn clean package you use]

Sh use for Shell Command

```
10
11 stage('Compile') {
12   steps {
13     sh "mvn compile"
14   }
15 }
16
```

3. **Test cases** : If Test cases are Written are Working Fine

In case Test cases are Pass You Can use

```
17 stage('Test') {
18   steps {
19     sh "mvn test"
20   }
21 }
22
```

In case **Test cases are Failing** and You Still want to Test your Application we can **skip the Test cases**

```
16
17 stage('Test') {
18   steps {
19     sh "mvn test -DskipTests=true"
20   }
21 }
22
```

In cases your Test cases are Fixed you can Remove **-Dskiptest=true**

4. After that I Perform **trivy File Scan**

For trivy there is **not Specific plugin** as of Now For that you need to Directly install on **Virtual machine**

Google it : Trivy install ubuntu You get commands to **install it** once you install here it will be Directly Available in here

To **check install it or not** Just Run command

\$ Trivy --version

```
23 stage('Trivy FS Scan') {
24   steps {
25     sh "trivy fs --format table -o fs-report.html ."
26   }
27 }
28
```

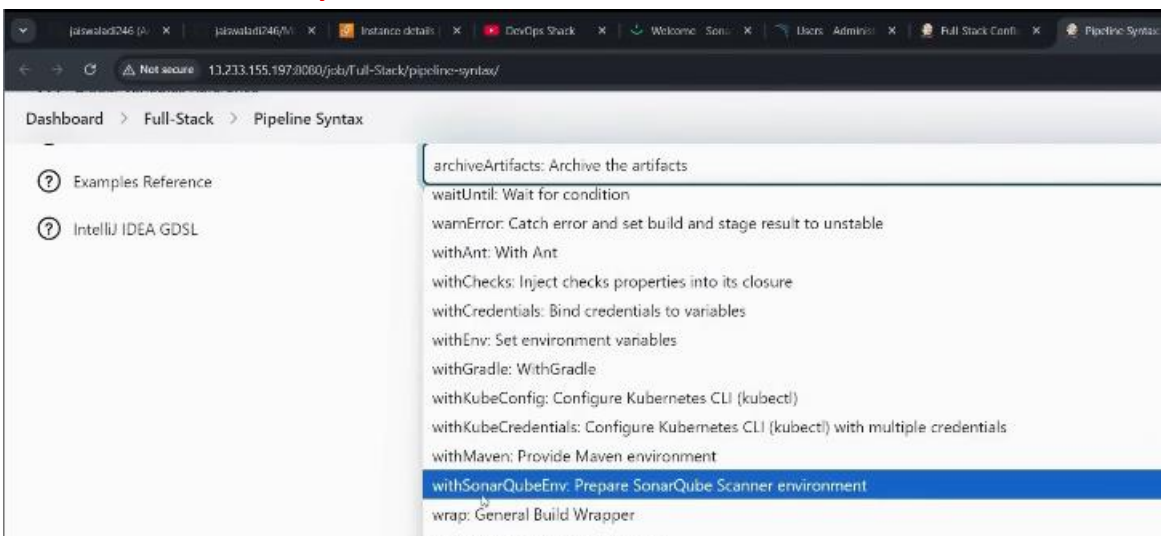
Trivy fs Because you want to **Scan file system** and you Need to **define** in which **Formate** you want your **Report (Table)**

- **-o** means you want **fs-report.html** formate

- After that you provide **Directory Location** Here we provide **.** (**Dot**) **Current Directory**

Next step :

5. **SonarQube Analysis :**



Server authentication token :

Sonar-token

Generate Pipeline script : copy that and modify it as you know that we already configure server sonarqube if I want to **publish** report to **server** I can simply provide the name of server (**'sonar'**)

First we call scanner **Executable file** so inside **home directory** we have folder called **/bin/sonar-scanner** executable file how we call it **\$SCANNER_HOME**

Now we need to add **Certain parameter**

You have to Connect with Jenkins + Sonarqube (with Credentials PAC)

```

stage("Sonarqube Analysis "){
    steps{
        withSonarQubeEnv('Sonar') {
            sh "$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Multitier \
            -Dsonar.projectKey=Multitier -Dsonar.java.binaries=target"
        }
    }
}

```

6. BUILD My Project to get Artifact JAR inside the Local Target folder

```

44
45 stage('Build') {
46     steps {
47         sh "mvn package -DskipTests=true"
48     }
49 }
50

```

7. Now I want My Artifact to Publish to Nexus

Go to **Nexus Dashboard** : You See Two repository

Name ↑	Type	Format	Blob Store	Status
maven-central	proxy	maven2	default	Online - Ready
maven-public	group	maven2	default	Online
maven-releases	hosted	maven2	default	Online
maven-snapshots	hosted	maven2	default	Online

First I **copy** Maven-release URL

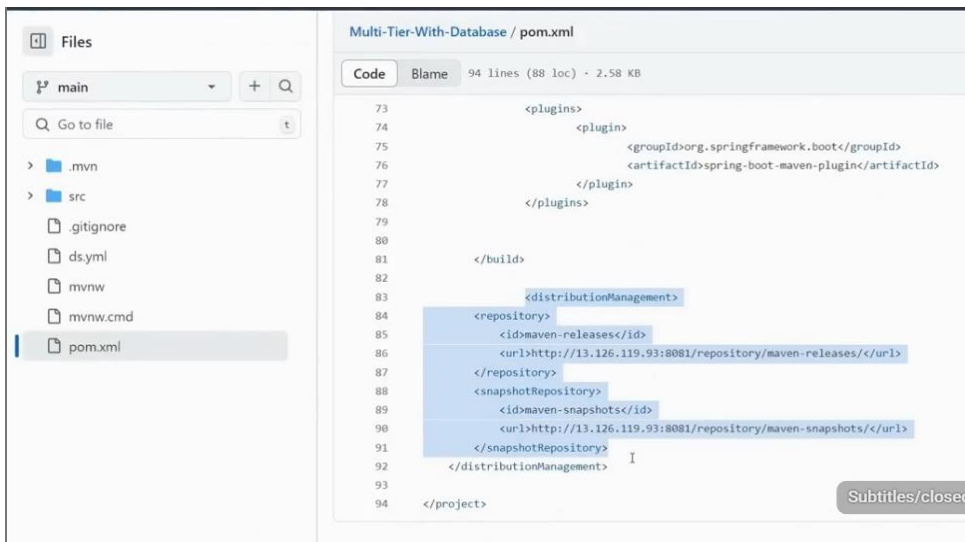
Copy to clipboard: Ctrl-C, Enter

Use your repository's direct URL (shown below) to connect other tools to your repository. For more information, see our Maven-specific help documentation.

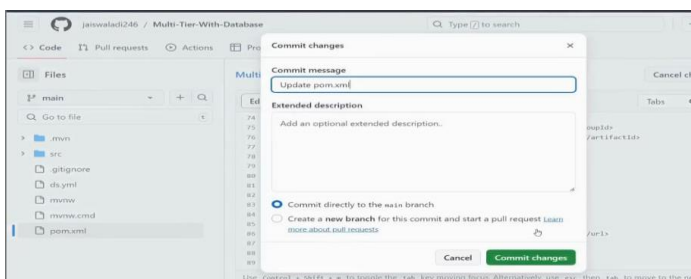
<http://310193578081/repository/maven-releases/>

Close

Go inside **Github + Pom.xml**



You should be able to find these Section here we provide **Repo URL** and **Maven snapshot URL**

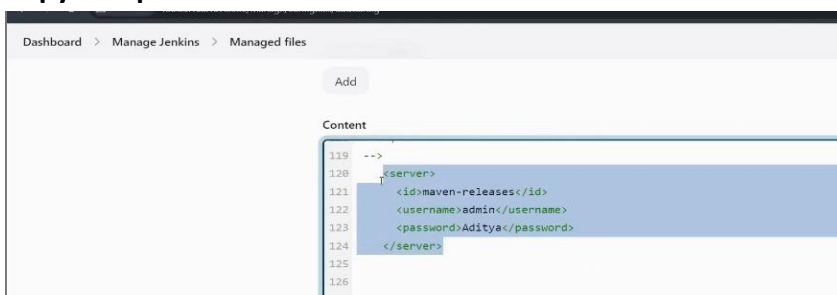


Now we have to provide **Credentials to Access** it Then we are able to **Publish your artifact**
Jenkins + Manage Jenkins + Managed files + Add new config + Global Maven Settings.xml

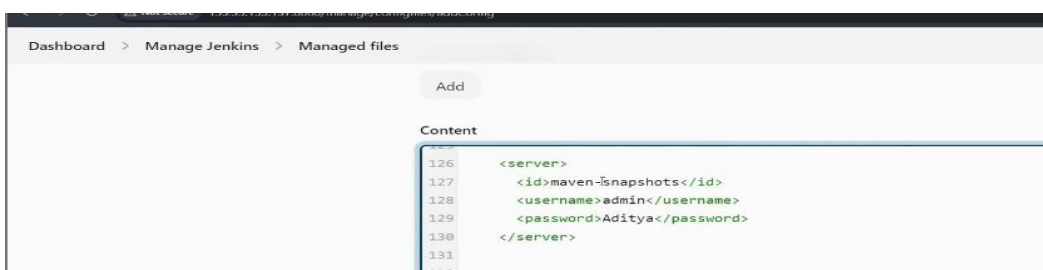


Here we want to **find section** called as **Server**.

Copy and past it here becasues we want two section here



Another section



Mvn deploy this basically **build artifact** and **deploy it to Nexus**

```

50
51 stage('Publish To Nexus') {
52     steps {
53         withMaven(globalMavenSettingsConfig: 'settings-maven', jdk: '', maven: 'maven3', mavenSettingsConfig: 'settings-maven') {
54             sh "mvn deploy -DskipTests=true"
55         }
56     }
57 }

```

Again test cases we skip it

The screenshot shows the Jenkins Pipeline Syntax Snippet Generator interface. On the left is a sidebar with a list of links: Snippet Generator, Declarative Directive Generator, Declarative Online Documentation, Steps Reference, Global Variables Reference, Online Documentation, Examples Reference, and IntelliJ IDEA GDSDL. The main area is titled 'Overview' and contains a description of the Snippet Generator. Below this is a 'Steps' section with a 'Sample Step' dropdown menu showing 'withDockerRegistry: Sets up Docker registry endpoint'. A 'Generate Pipeline Script' button is visible, and below it, a preview of the generated Pipeline Script code is shown.

Credentials + username and password

The screenshot shows the 'Jenkins Credentials Provider: Jenkins' form. It includes a checkbox for 'Treat username as secret', a 'Password' field, an 'ID' field with the value 'docker-cred', and a 'Description' field with the value 'docker-cred'.

Generate Pipeline Script.

Next step : Docker **Build Image**

Add credentials : Docker inside jenkins

```

58
59 stage('Docker Build Image') {
60     steps {
61         script {
62             withDockerRegistry(credentialsId: 'docker-cred', toolName: 'docker') {
63                 sh "docker build -t adijaiswal/bankapp:latest ."
64             }
65         }
66     }
67 }
68

```

Next stage : **Scan Docker image** :

```

69 stage('Trivy Image Scan') {
70     steps {
71         sh "trivy image --format table -o fs-report.html adijaiswal/bankapp:latest"
72     }
73 }
74

```

Next stage : **Push Docker image**

```

74
75 stage('Docker Push Image') {
76     steps {
77         script {
78             withDockerRegistry(credentialsId: 'docker-cred', toolName: 'docker') {
79                 sh "docker push adijaiswal/bankapp:latest"
80             }
81         }
82     }
83 }

```

Make sure to remove (dot) at end of latest

Next stage: In **Kubernetes Side :**

Best Option we go with using **RBAC (Role base access control)**

so that Not **Everyone having Permission** to perform their **Deployment**
only Specific account having Permission to perform deployment.

We are **Setup that Account** having **Permission** .

Now we go back to Server where we communicating with EKS cluster

Here I am create Directory RBAC

```

ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$ mkdir RAC
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$

```

```

ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$ ls
RBAC main.tf output.tf terraform.tfstate variables.tf
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform$ cd RBAC/
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$

```

We have to **Create Namespace: Webapps**

First we create **Service account**

```

ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl create namespace webapps
namespace/webapps created
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ vi svc.yml

```

```
Games Sessions View Split MultiExec Tunneling Packages Settings Help
7. Server
8. Nexus
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: webapps
~
~
~
```

```
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl create namespace webapps
namespace/webapps created
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ vi svc.yml
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl apply -f svc.yml
```

Service account Created

What is Service account ?

In Kubernetes, a ServiceAccount provides an **identity** for a **pod**. It allows the pod to authenticate with the Kubernetes API and **perform actions based on the permissions granted to it**.

#

Why Does Jenkins Need a ServiceAccount ?

If Jenkins deploys applications to Kubernetes, it needs permission to:

- Create Deployments
- Create Services
- Create Pods
- Update resources
- Delete resources

Instead of using **admin credentials**, Jenkins uses a dedicated ServiceAccount.

#

Now **we create ROLE** than **assign Role** to newly created **service account**.

[Create , update, delete, list, patch, watch]

```
serviceaccount/jenkins created
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ vi role.yml
```

Past it code.

```
172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl apply -f role.yml
authorization.k8s.io/app-role created
172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$
```

Now **Role** is created now we **assign to Service Account** we **bind** this to **service account**.

```

1  ### Bind the role to service account
2
3  ```yaml
4  apiVersion: rbac.authorization.k8s.io/v1
5  kind: RoleBinding
6  metadata:
7    name: app-rolebinding
8    namespace: webapps
9  roleRef:
10   apiGroup: rbac.authorization.k8s.io
11   kind: Role
12   name: app-role
13  subjects:
14   - namespace: webapps
15     kind: ServiceAccount
16     name: jenkins
17  ```

```

```

ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ vi bind.yml
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ Kubectl apply -f bind.yml
Command 'Kubectl' not found, did you mean:
  command 'kubectl' from snap kubectl (1.30.4)
See 'snap info <snapname>' for additional versions.
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl apply -f bind.yml

```

Now service account name: Jenkins having permission to perform everything

One last thing.

we want to do in order to use Jenkins for deployment.

Secrets :

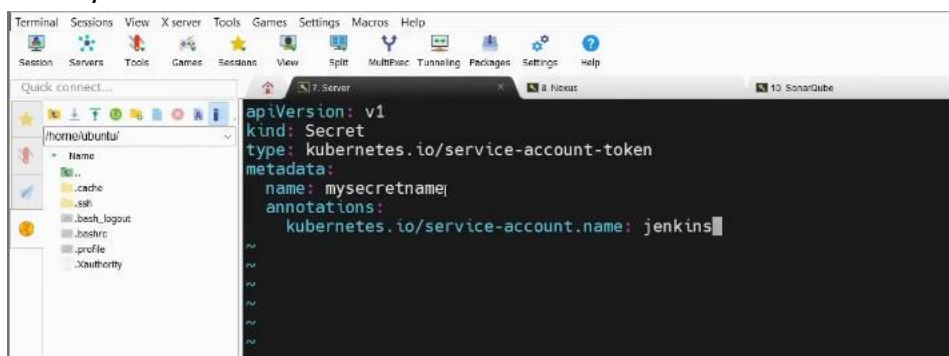
we need to Create Token for jenkins.

Our service account name is **jenkins**

I am going to say that secret is created in **webapps** namespace.

Now we get that token out of it

Vi sec.yml



```

apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: mysecretname
  annotations:
    kubernetes.io/service-account.name: jenkins

```

```

FullStack-Blogging-App/EKS_Terraform/RBAC$ vi sec.yml
FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl apply -f sec.yml -n webapps

```

I'm saying that these **secret** should be created inside **webapps** namespace

out of it

```
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ vi sec.yml
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl apply -f sec.yml -n webapps
secret/mysecretname created
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl describe secret mysecretname -n webapps
```

```
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl  
secret/mysecretname created  
ubuntu@ip-172-31-38-204:~/FullStack-Blogging-App/EKS_Terraform/RBAC$ kubectl  
Name: mysecretname  
Namespace: webapps  
Labels: <none>  
Annotations: kubernetes.io/service-account.name: jenkins  
kubernetes.io/service-account.uid: 9d810ebe-44cf-48c9-9667-5ba  
  
Type: kubernetes.io/service-account-token  
  
Data  
====  
ca.crt: 1107 bytes  
namespace: 7 bytes  
token: eyJhbGciOiJSUzI1NiIsImtpZCI6IlVfYnFsbnVowR3R4S3pVN0E0SU41T0Ribloy  
ldGVzLjNlcnZpY2YhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zXjZaZWNLWNjY3VudC9uYWwlc3R  
wNlYWNjb3VudC9zZWNYZXQubmFtZSI6Im15c2VjcmbV0bmFtZSI6Im15c2VjcmbV0ZXMuaW8vc2Vyc  
qZW5raW5ziIiwia3ViZXJuZXRlcy5pby9zXjZaZWNLWNjY3VudC9zXjZaZWNLWFwiY291bnQuZWI  
Tk3NGZhIiwic3ViOiJhc3lzdgVtOnNlcnZpY2YhY2NvdW50OndYmFwcHM6amVua2lucyJ9.L0Jj  
2EI1J2EGjFWiGDlBQObT5pscbsvYOAKxYZeb-UrCB7upMBirAY9BACW-Ucv2vcXqGgbt_BRzTS5nW  
Ip07rqMYf3WEfbPD2jRmoUtVg2pYfEbTGws9jwx7dgvEW5a3y0-TNyMu0GS5ET97PM5rytwM1o  
xcg6ruV_45gtfrBCCKBAWEFMvQ OPQ7v3Uvkcdp0vpugskgtw
```

Go to Jenkins + Configure + pipeline syntax

Pipeline Syntax

- archiveArtifacts: Archive the artifacts
- withGradle: withGradle
- withKubeConfig: Configure Kubernetes CLI (kubectl)**
- withKubeCredentials: Configure Kubernetes CLI (kubectl) with multiple credentials
- withMaven: Provide Maven environment
- withSonarQubeEnv: Prepare SonarQube Scanner environment

Credentials :

Kind : **Secret text : add token**

Jenkins Credentials Provider: Jenkins

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Secret

.....

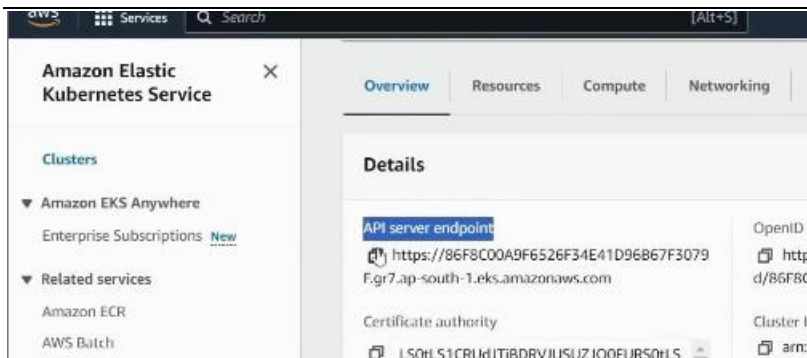
ID ?

k8-token

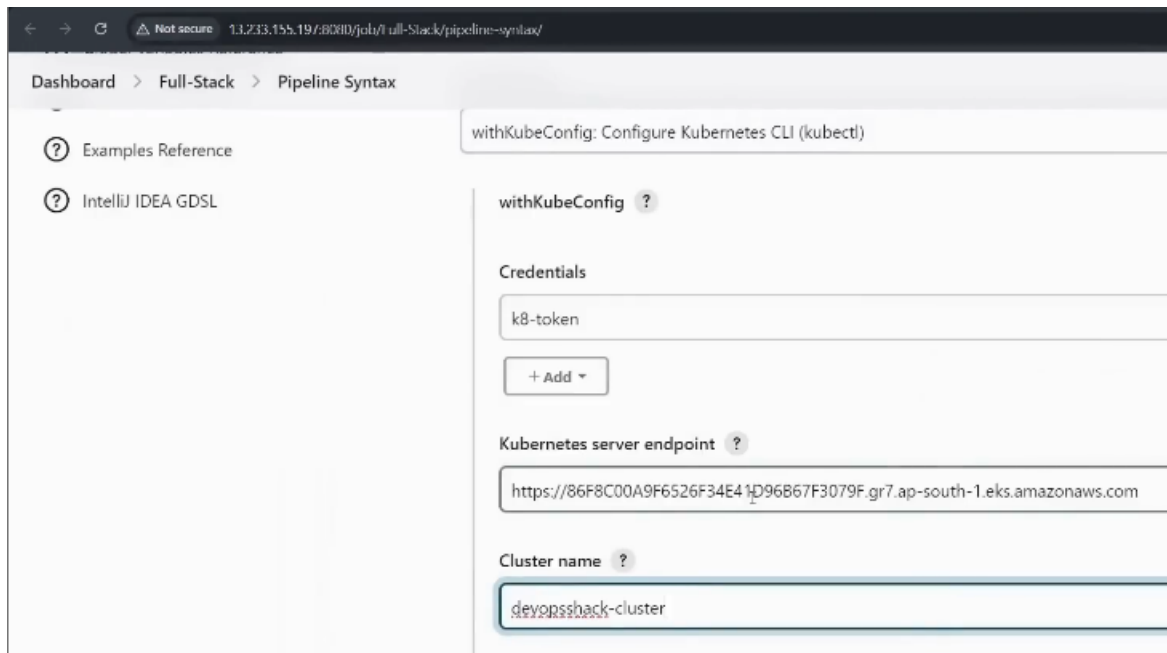
Description ?

k8-token

How we get server endpoint we setup inside EKS



There you can see **cluster name**



Namespace : webapps

Generate pipeline script .

Lets Understand : YAML files [**ds.yaml**] [Deployment mysql + service mysql]

We added **two environment variable** .

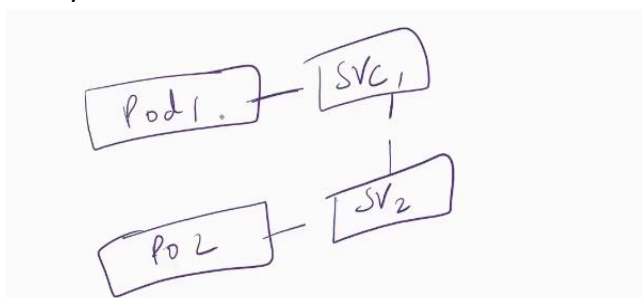
We using root username by default.

And we **setup password** for that Root passowrd:

Database name: **bankapp**

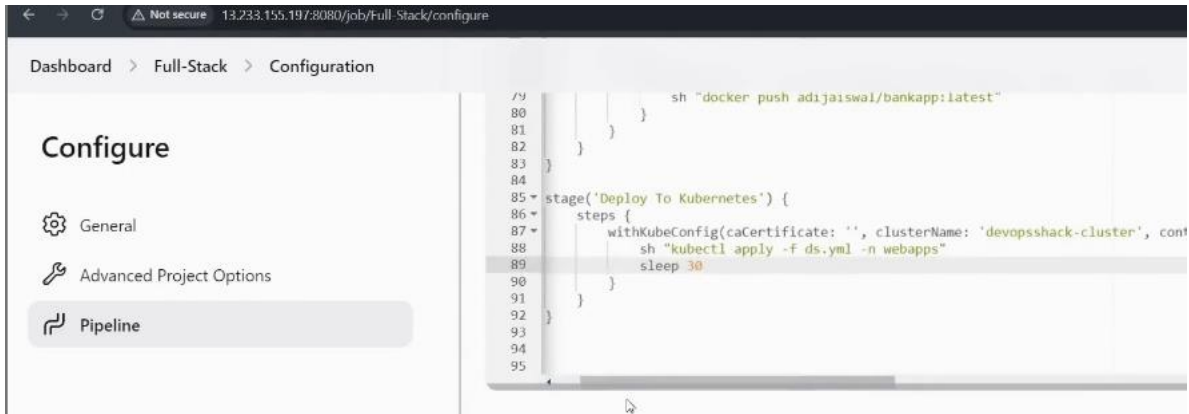
Than only we able to connect.

Always remember



If I say pod1 and pod 2 communicate with IP if I say pod1 get **restarted** or recreated than **ip** will **change** that why **we use Service that does not change**.

Deployment pod + Service pod



Why 30 sec because Deployment take time

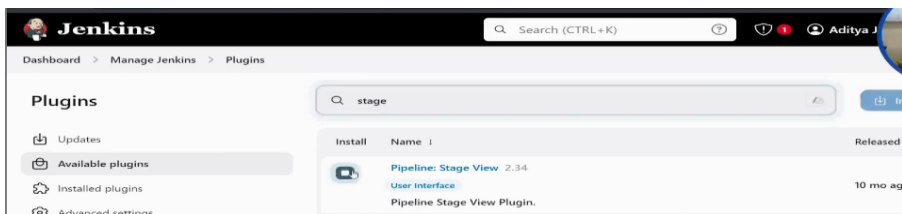
Next step : **Verify Deployment**



Save it Apply

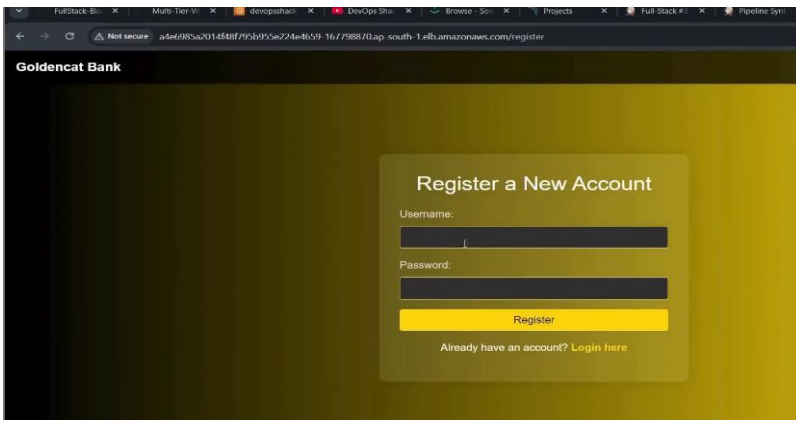
Build Now :

IF you **don't see Stages in dashboard** you have to add it : **Plugin** --- > Pipeline : **Stage View**



Kubectl get svc -n webapps

Lets **access thes application** using **load balancer URL**



Banking app here we add money + transfer money to + create new user from 1 user we send 2 user money and check transaction money also. **Done work fine application + DB connected**

How we Configure custom **Domain+ SSL certificate**.

Custom Domain :

As of Now we are **having Load balancer URL** that means

Loadbalancer is **Not Secure** Website ← HTTP that means What we are going to Do map **Custom Domain**

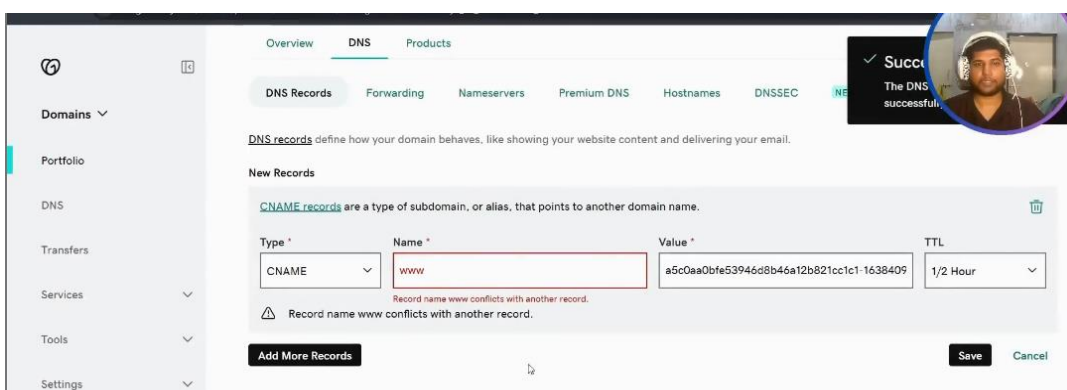
How we do that first of all you need to **purchase a Domain** from **Godady** After that you have to make **Configuration Domain side**.

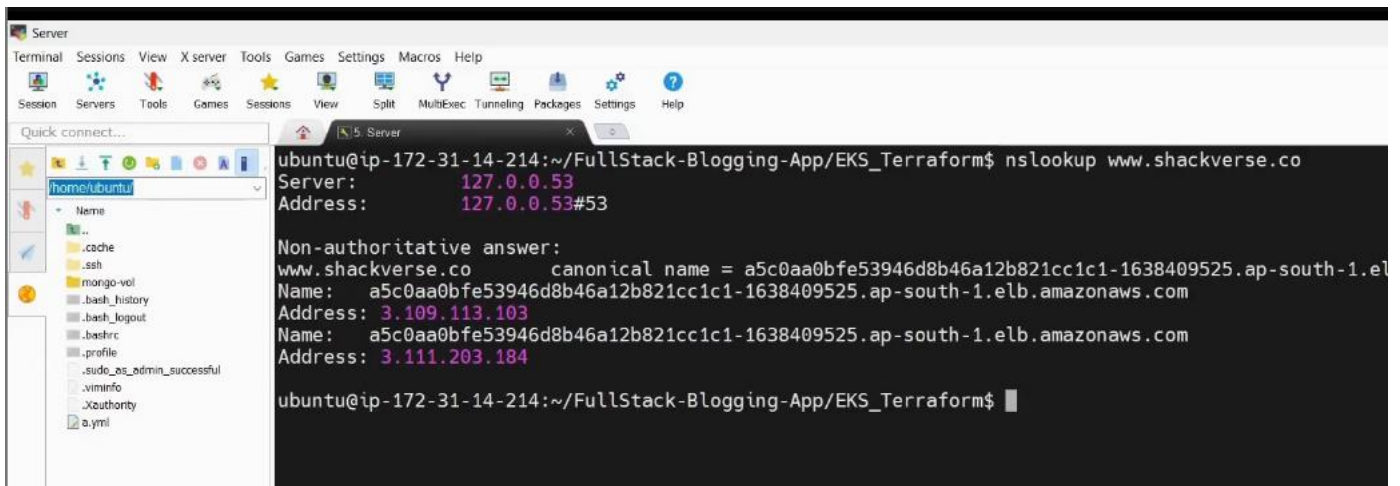
So that **my Domain will Point to loadbalancer URL**.

Go to Godady → Manage DNS →

So generally when you Deploy an Application 2 Cases happen .

1. Loadbalancer **URL** (Which we cannot remember) → **C Name**
2. External IP → **A Record**





```
Server
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help

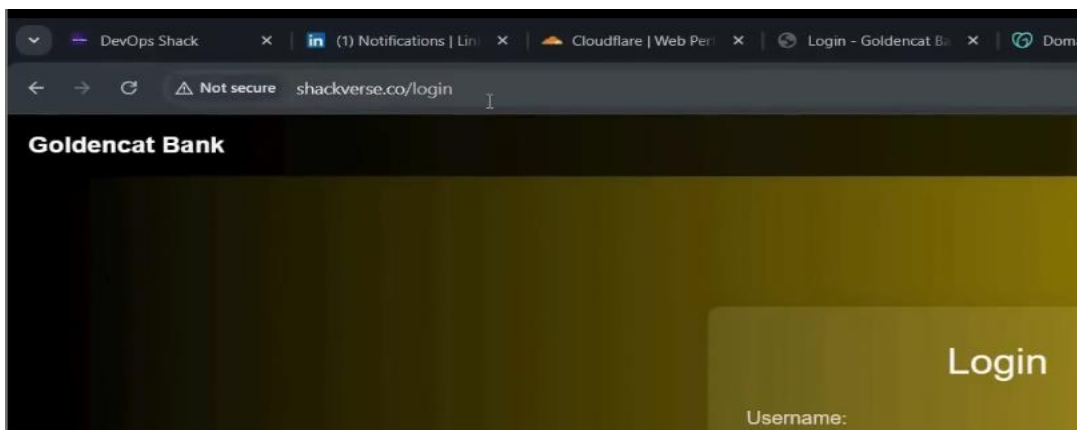
Quick connect...
home/ubuntu
Name
.cache
.ssh
mongo-vol
.bash_history
.bash_logout
.bashrc
.profile
.sudo_as_admin_successful
.viminfo
.xauthority
a.yml

ubuntu@ip-172-31-14-214:~/FullStack-Blogging-App/EKS_Terraform$ nslookup www.shackverse.co
Server:                127.0.0.53
Address:                127.0.0.53#53

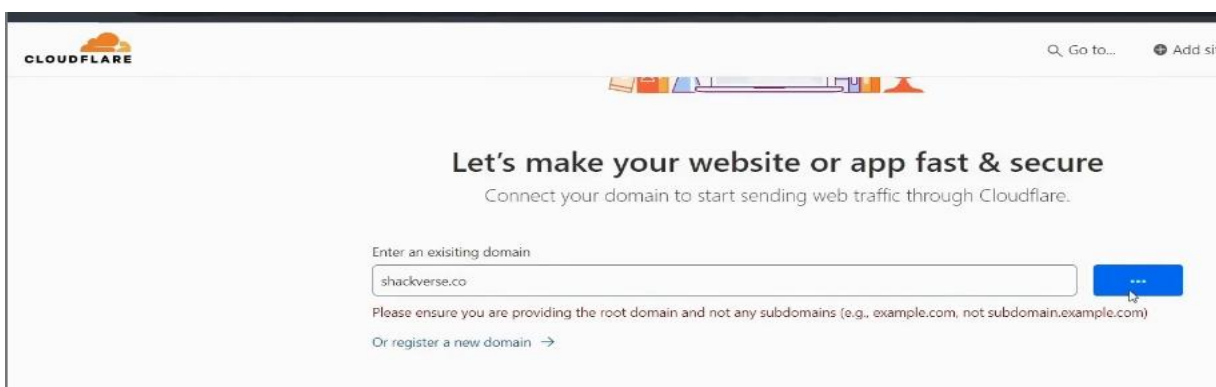
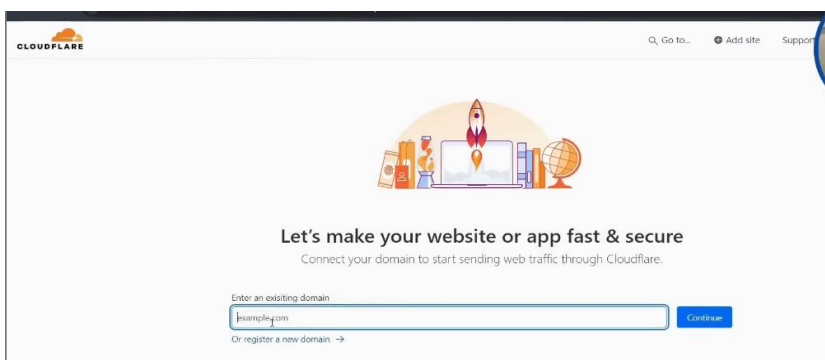
Non-authoritative answer:
www.shackverse.co      canonical name = a5c0aa0bfe53946d8b46a12b821cc1c1-1638409525.ap-south-1.elb.amazonaws.com
Name:                  a5c0aa0bfe53946d8b46a12b821cc1c1-1638409525.ap-south-1.elb.amazonaws.com
Address:               3.109.113.103
Name:                  a5c0aa0bfe53946d8b46a12b821cc1c1-1638409525.ap-south-1.elb.amazonaws.com
Address:               3.111.203.184

ubuntu@ip-172-31-14-214:~/FullStack-Blogging-App/EKS_Terraform$
```

Nslookup www.shackverse.co



But these is **Not secure** : Now we have to **Go to cloudflare**



Go at Bottom **Free**

You have to **update Your Nameservers** in **Godady** which **cloudflare** give you

3. Replace your current nameservers with Cloudflare nameservers

A. Find the nameservers section

B. Add both of your assigned Cloudflare nameservers:



barbara.ns.cloudflare.com [Click to copy](#)



rocco.ns.cloudflare.com [Click to copy](#)

Automatic HTTPS Rewrites

Automatic HTTPS Rewrites helps fix mixed content by changing "http" to "https" for all resources or links on your web site that can be served with HTTPS.

Create a [Configuration Rule](#) to customize these settings by hostname.



[API](#)

[Help](#)

Always Use HTTPS

Redirect all requests with scheme "http" to "https". This applies to all http requests to the zone.

This setting was last changed a few seconds ago

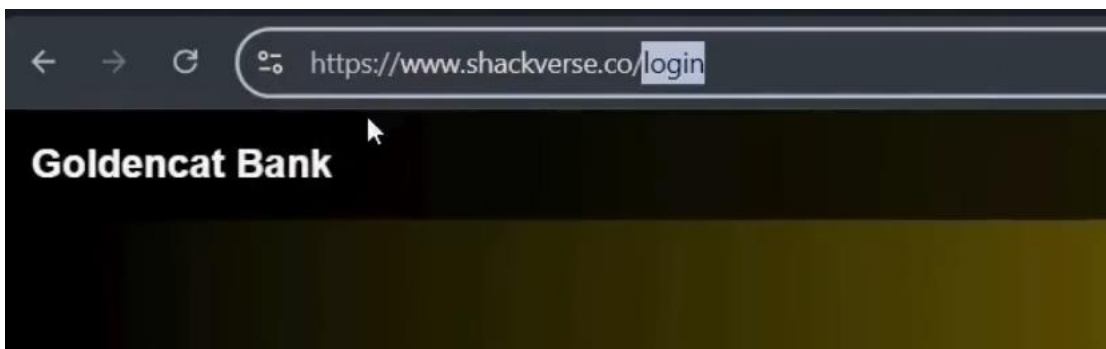


[API](#)

[Help](#)

[Previous](#)

[Save](#)



COMPLETE!